Project Genesis Engineering Principles

Project Genesis Research Repository

Generated: July 2026

Project Genesis Engineering Principles

This document outlines the core architectural and software engineering principles governing all subsystems of Genesis (the Simulation Engine and the Web Portal).

---

1. Single Subsystem Responsibility

Subsystems are strictly isolated:

- \textbfSimulation Engine (\textttworld/): Handles cellular and agent cognitive operations. It does not contain server or database integration logic.
- \textbfWeb Portal (\textttportal/): Handles ingestion validation, indexing, and visualization. It does not compute physics simulation steps.

2. The Experiment Package is the Contract

The exported ZIP archive is the absolute source of truth. All subsystems rely on the schema contracts defined in \texttshared/experiment_schema.json. We never compile database fields or client features that cannot be derived from this package contract.

3. Narrative Prioritized Over Analytics

The human chronicle of events (Founding, Expansion, Conflict, Decline, Collapse) is the entry gate to every experiment record. We present historical story records first to build emotional and context understanding, keeping data graphs decoupled inside the Observatory.

4. Scientific Reproducibility Above All

Simulations must be verifiable:

- All assets (configs, events, raw populations) are downloadable.
- Every experiment record includes citation support (BibTeX, APA) and a provenance tracker showing engine versions and staging logs.

5. Unified Schema Control

To prevent database, backend, and engine drifts, schema specifications are never duplicated. Shared types and JSON schemas live in the root \texttshared/ directory and are referenced directly during build steps.

6. Every Feature Must Answer a Research Goal

We do not add dashboards, telemetry charts, or spatial tracking features simply because the data exists. Every visualization must act as a target "Laboratory Instrument" resolving a specific research hypothesis.

7. Progressive Enhancement

We prioritize deep, robust implementations over feature accumulation. We build foundation layers (ingestion validation, transactional rollback, dynamic timeline chronicle) before visual complexity (interactive replays, animations, search graphs).